

ServiceNow Incident Management

Project Documentation Format

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident)
- **Team ID:** SWTID-2026-3114
- **Team Members:** List the team members and their respective roles.

Project Objective

The project focuses on implementing **Client Scripts and UI Policies** on the ServiceNow Incident form to improve user experience, enforce field behavior, validate Incident information, and reduce incorrect data entry.

2. Project Overview

Purpose

The purpose of this project is to dynamically control Incident fields based on user actions and Incident conditions. The solution ensures that important Incident information is validated and controlled without requiring unnecessary form submissions.

Features

- Automatically set **Urgency to High** when Impact is High.
 - Make the **Urgency** field read-only for High Impact Incidents.
 - Make **Assignment Group mandatory** when Impact is High.
 - Validate **Assigned To** before saving a High Impact Incident.
 - Prevent **State** changes through direct list editing.
 - Reverse UI Policy changes when the condition is no longer satisfied.
 - Display informational and error messages to users.
-

3. Architecture

User Interface

The solution uses the native **ServiceNow Incident Form and Incident List** as the user interface. Users can create, update, and manage Incident records through these interfaces.

Client-Side Logic

Client Scripts are implemented using **JavaScript** to perform actions based on user interactions.

The project uses:

- **onChange Client Script** – Automatically sets Urgency when Impact changes to High.
- **onSubmit Client Script** – Validates Assigned To before saving.
- **onCellEdit Client Script** – Prevents State changes through list editing.

UI Policy

A UI Policy named **High Impact Control** is configured on the Incident table.

When **Impact = High**, the policy controls the required field behavior, including making Urgency read-only and enforcing Assignment Group requirements.

Database

Incident information is stored in the **ServiceNow platform database**, including fields such as Impact, Urgency, Assigned To, Assignment Group, and State.

4. Setup Instructions

Prerequisites

- ServiceNow instance
- Appropriate administrative/configuration access
- Incident table access
- Access to **System UI** → **Client Scripts**
- Access to **System UI** → **UI Policies**

Configuration

1. Log in to the ServiceNow instance.

2. Navigate to **System UI** → **UI Policies**.
 3. Create the **High Impact Control** UI Policy for the Incident table.
 4. Configure the condition as **Impact is 1 – High**.
 5. Configure the required UI Policy Actions.
 6. Navigate to **System UI** → **Client Scripts**.
 7. Create the required Client Scripts:
 - Auto set urgency for high impact
 - Prevent save if Assigned To missing
 - Prevent state change via list edit
 8. Save and activate the configurations.
 9. Test the configurations using Incident records.
-

5. Configuration Structure

The project consists of the following ServiceNow configurations:

UI Policy

- **High Impact Control**
 - Table: Incident
 - Condition: Impact is High
 - Controls field behavior based on the condition.

UI Policy Action

- **Urgency**
 - Read-only when Impact is High.
- **Assignment Group**
 - Mandatory when Impact is High.

Client Scripts

Client Script	Type	Purpose
Auto set urgency for high impact	onChange	Sets Urgency to High when Impact is High.
Prevent save if Assigned To missing	onSubmit	Prevents saving a High Impact Incident when Assigned To is empty.
Prevent state change via list edit	onCellEdit	Prevents State changes through direct list editing.

6. Running / Executing the Application

The project does not require a separate local application server.

To execute the solution:

1. Log in to the ServiceNow instance.
2. Open the **Incident** module.
3. Create or open an Incident.
4. Change **Impact** to High.
5. Verify the automatic Urgency update and field controls.
6. Test Assigned To validation.
7. Open the Incident list and test State list editing.

7. Configuration Documentation

The project does not expose external REST APIs or application-specific endpoints.

The implementation uses native ServiceNow functionality:

- Client Scripts
- UI Policies
- UI Policy Actions
- GlideForm (**g_form**)
- Incident table fields

8. Access Control & Authorization

The solution is implemented within the ServiceNow platform and follows the access permissions and roles configured for the ServiceNow instance.

Only authorized users with appropriate configuration access can create or modify Client Scripts and UI Policies.

The implemented Client Scripts also prevent unintended Incident State changes through direct list editing.

9. User Interface

The project uses the standard ServiceNow Incident interface.

Screenshots can be included for:

- Incident form before applying the configuration.
- Impact set to High.
- Urgency automatically set to High.
- Urgency becoming read-only.
- Assignment Group mandatory behavior.
- Assigned To validation error.
- State list-edit blocking message.

Screenshots: Add the corresponding implementation screenshots from the ServiceNow instance here.

10. Testing

The implemented configuration was tested using multiple Incident scenarios.

Test Case	Expected Result	Status
Set Impact to High	Urgency is automatically set to High.	Passed
High Impact Incident	Urgency becomes read-only.	Passed
High Impact without Assigned To	Save is prevented and error message is displayed.	Passed
Change Impact from High to another value	UI Policy changes are reversed.	Passed
Edit State from Incident list	State change is blocked and warning is displayed.	Passed
Valid Incident form update	Incident is successfully saved.	Passed

11. Screenshots / Demo

Implementation screenshots are included in the project repository to demonstrate the configuration and testing results. The following are the implementation screenshots

1. UI Policy – **High Impact Control**
2. UI Policy Action – **Assignment Group**
3. UI Policy Action – **Urgency**
4. Client Script – **Auto set urgency**
5. Client Script – **Assigned To validation**
6. Client Script – **State list edit restriction**
7. High Impact Incident – **Urgency automatically updated**
8. Assigned To validation message
9. State list-edit blocking message
10. Successful Incident update

GitHub Repository:

<https://github.com/sharvena06/Implement-Client-Script-UI-Policy-Incident-.git>

12. Known Issues

- The configuration depends on the standard ServiceNow Incident fields being available.
 - Client-side validation can be bypassed through methods outside the configured user interface, so additional server-side controls may be required for stricter enforcement.
 - The current implementation is specifically designed for the Incident table and configured business conditions.
-

13. Future Enhancements

- Extend dynamic field controls to additional Incident fields.
 - Add more condition-based UI Policies.
 - Implement additional server-side validation where required.
 - Add automated testing for Client Scripts and UI Policies.
 - Extend similar controls to other ServiceNow tables.
 - Provide more detailed user guidance and notifications.
 - Integrate additional Incident management automation using ServiceNow Flow Designer.
-

Project Summary

The **Implement Client Script & UI Policy (Incident)** project improves ServiceNow Incident management by combining **Client Scripts, UI Policies, and UI Policy Actions**. The implementation provides automatic field updates, validation, controlled field behavior, and protection against incorrect list-based updates, resulting in more consistent and accurate Incident records.